

Introducción

El presente documento describe la arquitectura del sistema “Historia Interactiva IA”, una plataforma orientada a la generación dinámica de narrativas mediante técnicas de inteligencia artificial donde las decisiones del usuario determinan el curso y la dificultad de la historia. La aplicación ofrece una experiencia interactiva en la que el lector deja de ser un mero observador y participa activamente en la construcción del relato: tras una breve personalización inicial, el motor de IA genera escenas y propone al menos tres opciones significativas en cada punto de decisión; las elecciones del usuario no solo condicionan la trama sino que alteran el grado de reto de las situaciones futuras mediante un mecanismo de adaptación de dificultad. Además de la experiencia narrativa en tiempo real, la plataforma incorpora funcionalidades de gestión de cuentas, persistencia de historias para reanudación posterior y un panel de progreso que registra los contextos clave y las decisiones tomadas, permitiendo al usuario revisar y continuar su viaje narrativo cuando lo desee.

Desde la perspectiva organizativa y técnica, este documento sitúa el sistema en su entorno: especifica los límites del producto, identifica los actores principales implicados en su uso y operación, y clarifica las interacciones con servicios externos como sistemas de autenticación, almacenes de datos y posibles servicios de IA en la nube. Está pensado como una guía viva para desarrolladores, diseñadores, evaluadores y administradores, de forma que todos los participantes del proyecto compartan una visión coherente sobre las responsabilidades funcionales, las dependencias tecnológicas y las expectativas no funcionales. El enfoque de la arquitectura busca equilibrar la flexibilidad necesaria para iterar sobre los modelos de generación y ajuste de dificultad con la solidez requerida para proteger datos de usuarios, garantizar tiempos de respuesta razonables y facilitar el mantenimiento y la evolución del producto.

La justificación de esta arquitectura radica en cuatro necesidades concretas: primero, proporcionar una experiencia narrativa coherente y adaptativa que mantenga al usuario comprometido; segundo, permitir que el equipo de desarrollo implemente y despliegue componentes de IA y almacenamiento de forma modular y trazable; tercero, ofrecer mecanismos de seguridad y protección de datos que cumplan con buenas prácticas; y cuarto, dar soporte a la evolución del sistema para incorporar nuevos géneros, mejoras en el motor de IA o integración con terceros. Para lograrlo, la documentación arquitectónica que aquí se presenta adopta un enfoque basado en viewpoints conforme a la norma ISO/IEC/IEEE 42010, de modo que cada punto de vista —contexto, composición, lógico, de

interfaz, de estructura, de interacción, de dinámica de estados y de algoritmo— quede claramente definido, con sus modelos, reglas de correspondencia y métodos de análisis.

Finalmente, este documento se concibe como un instrumento de comunicación técnica y de gobernanza: permitirá validar que las decisiones de diseño satisfacen los requisitos funcionales y de calidad, servirá como referencia para la implementación y las pruebas, y facilitará la toma de decisiones futuras sobre escalabilidad, despliegue y mantenimiento. Su estructura modular y su orientación a viewpoints facilitan la actualización incremental de la arquitectura a medida que el proyecto avance y surjan nuevas necesidades, sin perder la trazabilidad entre requisitos, diseño y entrega.

Objetivo

El objetivo de este documento es definir, explicar y formalizar la arquitectura del sistema “Historia Interactiva IA” de tal manera que sirva como referencia única y autoritativa para todos los actores del proyecto: describe la estructura del sistema, las relaciones entre sus componentes, los modelos de interacción con usuarios y servicios externos, las reglas de correspondencia entre vistas y los criterios de calidad que deben cumplirse, con el fin de guiar el desarrollo, el despliegue y la evolución del producto. A través de una documentación basada en viewpoints conforme al anexo B de la norma ISO/IEC/IEEE 42010, se pretende proporcionar una base clara para la implementación técnica, facilitar la verificación y validación frente a los requisitos y asegurar que las decisiones arquitectónicas soporten la adaptabilidad narrativa, la seguridad de los datos, el rendimiento operativo y la facilidad de mantenimiento del sistema a lo largo del tiempo.

Context Viewpoint – Proyecto: Historia Interactiva IA

Viewpoint Name

Context Viewpoint

El Punto de Vista del Contexto. Se centra en mostrar el sistema como una “caja negra” dentro de su entorno, resaltando las relaciones con actores humanos, sistemas externos y servicios de soporte.

2. Resumen

El Context Viewpoint se encarga de describir el sistema de manera global, identificando los límites que lo separan de su entorno, los actores externos con los

que interactúa y los principales flujos de información que se intercambian. En el caso del sistema “Historia Interactiva IA”, este viewpoint es fundamental porque permite comprender la interacción entre la aplicación, los usuarios finales y los servicios que soportan la generación dinámica de narrativas.

Al ser la primera vista en la arquitectura, ofrece a los interesados una visión clara de “quién usa el sistema, cómo lo usa y qué dependencias externas existen”, estableciendo el marco de referencia para las vistas más detalladas que se desarrollarán posteriormente.

3. Preocupaciones y atributos de calidad de la arquitectura

A continuación, se detallan las preocupaciones clave para los diferentes *stakeholders* del sistema, que han servido de guía para las decisiones arquitectónicas. Estas preocupaciones se abordarán a través de las diferentes vistas de la arquitectura.

Preocupaciones por *Stakeholder*

Usuarios Finales

- **Interactividad:** La experiencia narrativa debe ser fluida, con una alta capacidad de respuesta y una sensación real de que las decisiones del usuario tienen un impacto significativo y coherente en la historia.
- **Coherencia de la historia:** El motor de IA debe generar narrativas lógicas y consistentes, evitando saltos argumentales o incongruencias que puedan romper la inmersión del usuario.
- **Adaptabilidad:** La dificultad del relato debe ajustarse de forma dinámica a las elecciones del usuario, manteniendo un nivel de reto óptimo sin frustrar ni aburrir al lector.
- **Privacidad de datos:** La información personal y el historial de historias del usuario deben estar protegidos y ser manejados de forma segura, cumpliendo con las políticas de protección de datos.
- **Usabilidad de la interfaz:** La personalización inicial y el panel de progreso deben ser intuitivos y fáciles de usar, incluso para usuarios sin conocimientos técnicos.

Desarrolladores

- Entendimiento del sistema: La arquitectura debe ser clara y estar bien documentada para que los desarrolladores comprendan las interacciones con los servicios externos (IA, base de datos, autenticación).
- Modularidad de la IA: El componente de inteligencia artificial debe estar desacoplado del resto del sistema para permitir iteraciones, mejoras y sustituciones sin afectar a la funcionalidad principal.
- Evolucionabilidad: La arquitectura debe permitir la incorporación de nuevos géneros, modelos de IA y mejoras funcionales de manera incremental y con un impacto mínimo en los componentes existentes.
- Depuración: La capacidad de registrar y trazar los flujos de datos y la lógica del sistema es crucial para diagnosticar problemas en la generación de la historia o la gestión de la dificultad.

Administradores del Sistema

- Disponibilidad y rendimiento: El sistema debe ser robusto y capaz de manejar un alto volumen de usuarios concurrentes, garantizando tiempos de respuesta razonables.
- Escalabilidad: El sistema debe poder crecer de forma eficiente para soportar un aumento en el número de usuarios y la carga de procesamiento, especialmente en los servicios de IA y la base de datos.
- Seguridad: La plataforma debe contar con mecanismos de seguridad robustos para proteger los datos de los usuarios, evitar accesos no autorizados y asegurar que las interacciones con servicios externos sean seguras.
- Monitoreo: El sistema debe proporcionar herramientas e interfaces para monitorear el estado de los componentes, el uso de recursos y el comportamiento de la plataforma.

Equipo de Pruebas

- Trazabilidad: La arquitectura debe permitir rastrear los requisitos desde el diseño hasta la implementación, facilitando la creación de casos de prueba exhaustivos.
- Aislamiento de componentes: Los componentes deben poder ser probados de forma independiente para verificar su correcto funcionamiento y sus interacciones.

- Comportamiento de la IA: Es necesario un mecanismo para verificar y validar la coherencia y la adaptación de la dificultad que ofrece el motor de inteligencia artificial.

Cientes Interesados

- Viabilidad comercial: El diseño de la arquitectura debe ser escalable para acomodar el crecimiento del negocio, permitiendo un retorno de la inversión.
- Protección de la propiedad intelectual: El contenido generado por el sistema debe protegerse de usos comerciales no autorizados, lo que requiere considerar licencias y medidas técnicas.
- Innovación: La arquitectura debe ser lo suficientemente flexible para integrar nuevas tecnologías y servicios que mantengan el producto competitivo.

"anti-preocupaciones"

- Escalabilidad: Se aborda mediante una arquitectura de microservicios, que permite escalar componentes de forma independiente (ej. el servicio de generación de IA o la base de datos de historias).
- Actualización: La modularidad del diseño facilita la evolución del sistema. Los nuevos modelos de IA o mejoras se pueden implementar y desplegar con independencia del resto de la aplicación.

4. Stakeholders típicos y sus intereses

- Usuarios finales: Personas que interactúan con la plataforma.
 - Interés: Usabilidad, coherencia de la narrativa, adaptabilidad de la dificultad, privacidad de datos.
- Administradores del sistema: Personal encargado del mantenimiento y operación.
 - Interés: Disponibilidad, seguridad, monitoreo, gestión de cuentas.
- Desarrolladores: Equipo técnico encargado de la implementación y evolución del sistema.
 - Interés: Modularidad, integración con servicios externos, depuración.
- Equipo de Pruebas: Responsables de la validación y verificación del sistema.

- Interés: Trazabilidad de requisitos, prueba de componentes, validación del motor de IA.
- Clientes interesados (Negocio): Quienes evalúan el valor y viabilidad del producto.
 - Interés: Alcance funcional, potencial de crecimiento (escalabilidad), protección de propiedad intelectual.

5. Tipos de modelos (Model kinds)

El Context Viewpoint se documentará principalmente con:

Diagrama de Contexto.

Tablas de relaciones: Listado detallado de cada actor externo y los datos que intercambia con el sistema.

6. Reglas de correspondencia

- Correspondencia con actores externos: Todo actor externo y servicio descrito en el diagrama de contexto deberá tener una representación formal en las tablas de relaciones.
- Correspondencia con el sistema: El límite del sistema, tal como se define en esta vista, debe ser consistente con la composición interna de los componentes descrita en el Composiiton Viewpoint.
- Correspondencia entre vistas: Los flujos de datos identificados entre el sistema y los actores externos (por ejemplo, las credenciales del usuario o las opciones de la historia) deben ser detallados y expandidos en el Interaction Viewpoint y el Logical Viewpoint.

7. Operaciones sobre vistas

- Creación: A partir de requisitos funcionales y entrevistas con stakeholders, se construye el diagrama de contexto y la tabla de relaciones.
- Interpretación: Los interesados pueden usar esta vista para comprender qué papel cumple el sistema y cómo se conecta con su entorno.
- Análisis: Permite verificar que no se ha omitido ningún actor clave y que los flujos de información están bien delimitados.

- Diseño/Implementación: Sirve como base para definir interfaces de comunicación y dependencias externas a implementar.

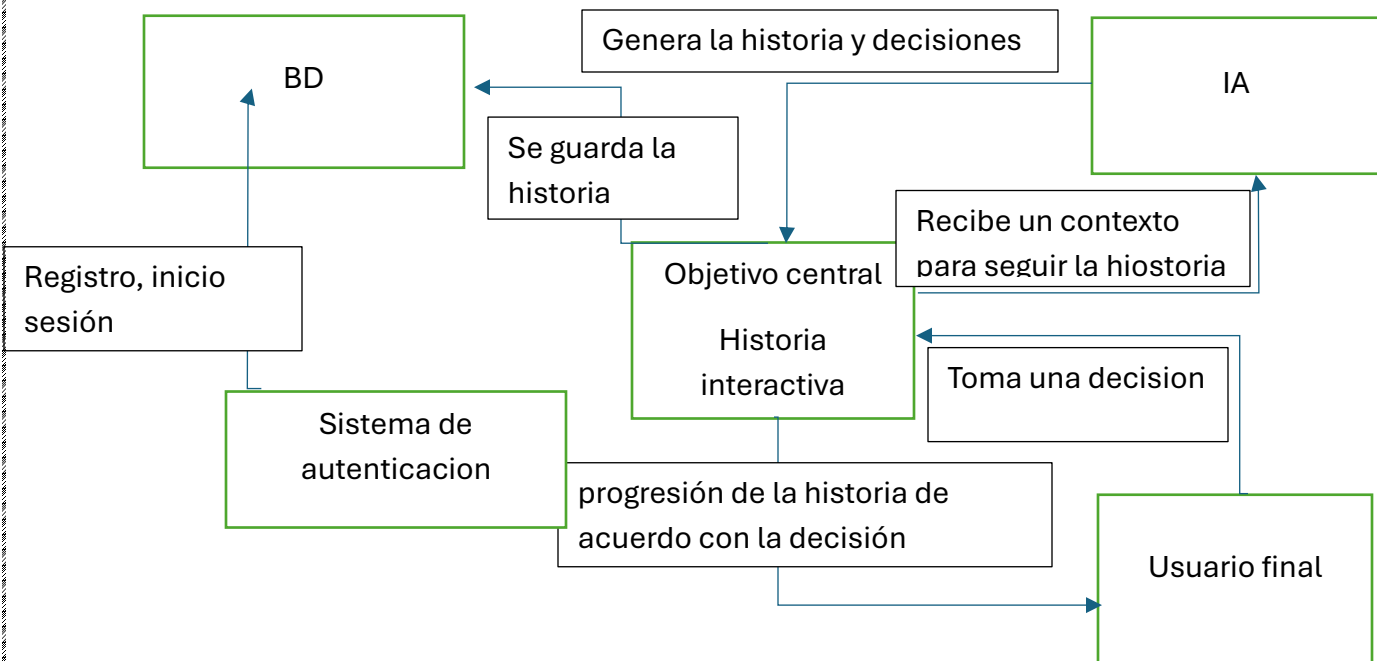
8. Ejemplo del sistema

En el caso del proyecto Historia Interactiva IA, el diagrama de contexto debe mostrar:

- Al usuario final que interactúa con el sistema a través de la interfaz web.
- Al sistema de autenticación (registro, inicio/cierre de sesión).
- Al motor de IA narrativa que genera las historias y decisiones.
- A la base de datos donde se guardan cuentas de usuarios, historias en progreso e historial.
- Al sistema externo de almacenamiento (si existe nube o backups).

Tipo de diagrama aquí: Diagrama de Contexto (*System Context Diagram*).

- Para qué sirve: visualizar de manera simple las fronteras del sistema y las interacciones principales con su entorno.
- Cómo elaborarlo: dibujar una caja central con el nombre del sistema (“Historia Interactiva IA”), rodeada de actores externos (usuarios, IA, base de datos, sistema de autenticación). Usar flechas para representar flujos de información (ejemplo: *Usuario* → *credenciales* → *Sistema* / *Sistema* → *opciones de historia* → *Usuario*).



9. Notas adicionales

- Este viewpoint debe mantenerse actualizado cada vez que se incorpore un nuevo servicio externo (ejemplo: integración con redes sociales o APIs de terceros).
- Es recomendable complementarlo con descripciones textuales para cada flujo, ya que el diagrama por sí solo puede ser demasiado abstracto.

10. Fuentes

- ISO/IEC/IEEE 42010:2011 — *Systems and software engineering — Architecture description*.
- Documentación del proyecto “Historia Interactiva IA”.
- Best practices en modelado de arquitectura (Kruchten, Rozanski & Woods, Clements et al.).

1. Nombre del Viewpoint

Composition Viewpoint

Este punto de vista se centra en la estructura interna del sistema y en cómo sus principales componentes se organizan para cumplir con la funcionalidad global. Permite entender de qué partes está formado el sistema, cómo se relacionan y cuáles son sus responsabilidades.

2. Resumen

El Composition Viewpoint describe el sistema Historia Interactiva IA desde la perspectiva de su descomposición en módulos o componentes principales. Mientras

Este viewpoint es esencial para identificar los bloques de construcción de la aplicación, sus límites, los flujos de información entre ellos y cómo cada componente contribuye a la generación, gestión y entrega de las historias interactivas.

3. Preocupaciones y anti-preocupaciones

Preocupaciones principales

- Modularidad: que el sistema esté dividido en componentes con responsabilidades claras y poco acoplamiento.
- Escalabilidad: que la adición de nuevos módulos (por ejemplo, un motor narrativo más complejo o nuevos métodos de autenticación) no afecte al resto del sistema.
- Mantenibilidad: que cada componente sea fácilmente actualizable de forma independiente.
- Integración fluida: que los módulos de IA, autenticación, base de datos e interfaz gráfica trabajen de manera conjunta sin redundancias ni conflictos.

Anti-preocupaciones

- No se detallan aquí los algoritmos internos de generación de historia (eso corresponde al Algorithm Viewpoint).
- No se entra en detalle sobre interacciones paso a paso con el usuario (eso corresponde al Interaction Viewpoint).

4. Stakeholders típicos

- Desarrolladores: requieren comprender la estructura modular para implementar, probar y mantener cada componente.
- Equipo de pruebas: necesitan conocer los módulos y sus límites para diseñar casos de prueba de integración.
- Administradores de sistema: interesados en saber cómo se despliegan los módulos en la infraestructura (por ejemplo, qué depende de la base de datos).
- Usuarios finales: aunque no ven directamente esta vista, se benefician de un sistema estable, modular y con componentes bien definidos.

5. Tipos de modelos (Model kinds)

Este viewpoint se representará principalmente con:

- Diagrama de Componentes (UML): muestra los módulos internos del sistema y sus relaciones.

6. Reglas de correspondencia

- Cada componente debe corresponder a una funcionalidad descrita en los requisitos del sistema.
- Los flujos de información deben ser consistentes con los flujos mostrados en el Context Viewpoint.
- Los límites entre módulos deben ser claros, diferenciando entre lo que pertenece al sistema y lo que son servicios externos.

7. Operaciones sobre vistas

- Creación: surge a partir de la identificación de funcionalidades críticas (autenticación, generación de historia, persistencia de datos, interfaz).
- Interpretación: permite a los interesados entender cómo se organiza internamente la aplicación.

- **Análisis:** facilita identificar redundancias, acoplamientos innecesarios o dependencias críticas.
- **Diseño/Implementación:** actúa como guía directa para la construcción de la arquitectura de software.

8. Ejemplo aplicado al sistema

Para el sistema Historia Interactiva IA, los principales componentes internos serían:

1. **Interfaz de Usuario (Frontend Web/App)**
 - Presenta las historias generadas y las opciones de decisión.
 - Permite registro, inicio/cierre de sesión y navegación entre menús.
 - Muestra el progreso del usuario y sus historias guardadas.
2. **Módulo de Autenticación**
 - Gestiona registro, login/logout y seguridad de las cuentas.
 - Implementa cifrado para proteger credenciales y datos personales.
3. **Motor de IA Narrativa**
 - Genera las historias dinámicas y decisiones en función de las elecciones del usuario.
 - Se comunica con el componente de personalización para adaptar género, tono y dificultad.
4. **Módulo de Personalización**
 - Recoge preferencias del usuario (género, nivel de dificultad, estilo narrativo).
 - Pasa parámetros al motor de IA narrativa.
5. **Módulo de Gestión de Progreso**
 - Controla el avance de cada historia en curso.
 - Almacena y recupera historias guardadas.
 - Permite reanudar historias incompletas.
6. **Base de Datos**

- Almacena usuarios, credenciales, configuraciones, historias generadas y su historial.
- Conecta con el módulo de gestión de progreso y el motor narrativo.

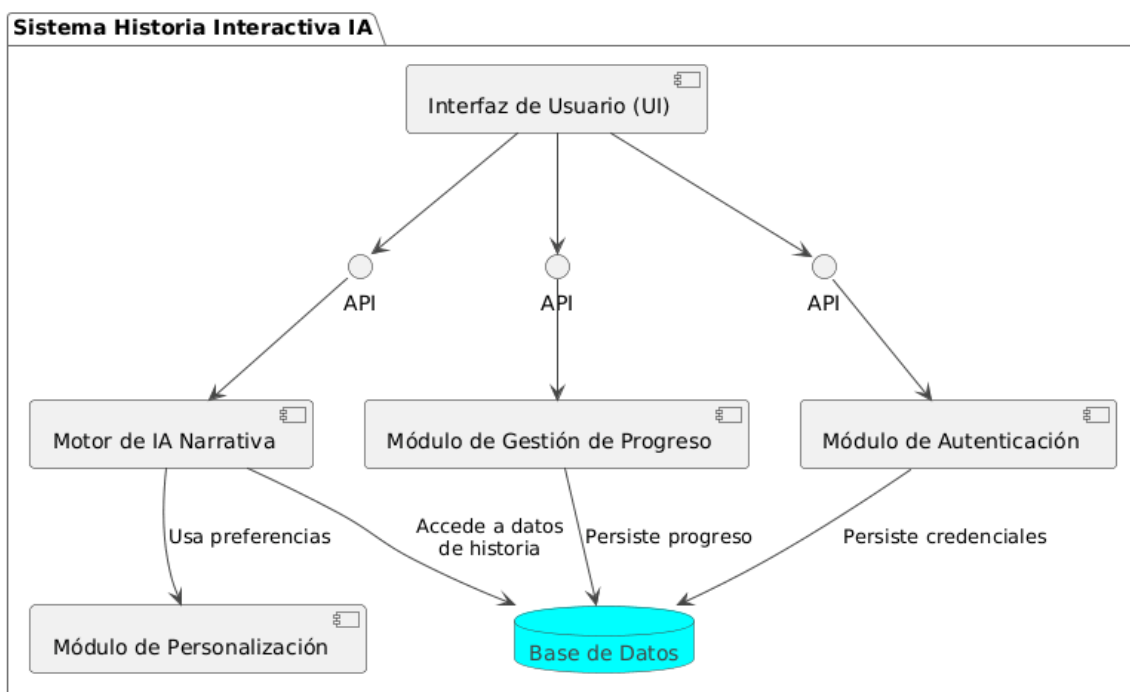
7. Módulo de Administración del Sistema

- Herramientas para que el administrador gestione usuarios y supervise la aplicación.
- Controla políticas de seguridad y disponibilidad.

9. Tipo de diagrama a elaborar

Diagrama de Componentes UML

- ¿Para qué sirve?
Para visualizar cómo se descompone el sistema en componentes internos, sus interfaces y conexiones.



1. Nombre del Viewpoint

Logical Viewpoint

Este punto de vista se centra en representar el sistema desde la perspectiva lógica de clases, objetos y relaciones, mostrando cómo se organiza la información y la funcionalidad a nivel conceptual. Es clave para vincular requisitos con elementos de diseño que posteriormente podrán traducirse en componentes físicos o módulos de software.

2. Resumen

El Logical Viewpoint describe la arquitectura de software de forma abstracta, mostrando qué entidades existen, qué responsabilidades tienen y cómo se relacionan entre sí. A diferencia del Composition Viewpoint, que muestra bloques de alto nivel (módulos y componentes), aquí se da un paso más hacia el diseño de software detallado, organizando las clases, sus atributos y operaciones, así como las asociaciones necesarias.

Este viewpoint es especialmente útil en el sistema Historia Interactiva IA, ya que las historias generadas dinámicamente, las decisiones del usuario, el progreso, la personalización y los usuarios registrados requieren de una estructura lógica clara que garantice coherencia, escalabilidad y mantenibilidad.

3. Preocupaciones y anti-preocupaciones

Preocupaciones principales

- Claridad en la estructura de datos: definir cómo se almacenan usuarios, historias, decisiones y progresos.
- Consistencia de relaciones: evitar redundancias o inconsistencias en los vínculos entre clases.
- Separación de responsabilidades: cada clase debe tener un rol claro y no mezclar múltiples funciones.
- Reutilización: asegurar que las clases sean reutilizables en diferentes contextos de la aplicación (ejemplo: personalización en varias historias).

Anti-preocupaciones

- No se describen aquí aspectos físicos de despliegue (eso corresponde a otros viewpoints como Resource Viewpoint).

- No se entra en detalle sobre procesos dinámicos (eso se verá en el State Dynamics Viewpoint).

4. Stakeholders típicos

- Desarrolladores: necesitan un diseño lógico claro para transformar los modelos en código.
- Equipo de pruebas: les permite entender qué clases existen y cómo se relacionan para generar casos de prueba unitarios.
- Arquitectos de software: validan la corrección de las relaciones y la adecuación del modelo lógico a los requisitos.
- Administradores del sistema: indirectamente, se benefician de un modelo lógico robusto que minimice errores en la gestión de datos.

5. Tipos de modelos (Model kinds)

- Diagrama de Clases UML: representa clases, atributos, operaciones y asociaciones.
- Diagrama de Objetos UML: muestra instancias concretas de las clases y cómo interactúan en un caso particular (ejemplo: un usuario con una historia personalizada en progreso).

6. Reglas de correspondencia

- Cada clase debe corresponder a un concepto clave del dominio (usuario, historia, decisión, progreso, personalización).
- Las relaciones deben ser consistentes con los flujos de información ya descritos en Context y Composition viewpoints.
- Los atributos deben reflejar datos reales necesarios para la aplicación, mientras que las operaciones deben estar alineadas con la funcionalidad descrita en los requisitos.

7. Operaciones sobre vistas

- Creación: se construye a partir de requisitos funcionales y entidades identificadas en el dominio narrativo.
- Interpretación: permite a los desarrolladores y arquitectos comprender cómo se organiza la lógica del sistema.
- Análisis: ayuda a verificar que no se omitió ninguna entidad clave y que las relaciones son coherentes.
- Diseño/Implementación: sirve como modelo directo para codificar las clases y sus interacciones.

8. Ejemplo aplicado al sistema

En el sistema Historia Interactiva IA, las principales clases serían:

- Usuario
 - Atributos: idUsuario, nombre, email, contraseña, edad.
 - Operaciones: registrarse(), iniciarSesion(), cerrarSesion(), personalizarHistoria().
- Historia
 - Atributos: idHistoria, título, contenido, estado (en curso, terminada).
 - Operaciones: generarHistoria(), guardarHistoria(), cargarHistoria().
- Decisión
 - Atributos: idDecision, textoOpcion, consecuencia.
 - Operaciones: elegirOpcion().
- Progreso
 - Atributos: idProgreso, idHistoria, idUsuario, porcentajeAvance, decisionesTomadas[].
 - Operaciones: actualizarProgreso(), mostrarResumen().
- IA Narrativa
 - Atributos: idMotor, version, parametrosGeneracion.
 - Operaciones: generarTexto(), generarOpciones(), adaptarDificultad().
- Personalización

- Atributos: idPersonalizacion, genero, nivelDificultad, tonoNarrativo.
- Operaciones: aplicarPreferencias(), modificarPreferencias().
- AdministradorSistema
 - Atributos: idAdmin, nombre.
 - Operaciones: gestionarUsuarios(), supervisarSistema().

9. Tipo de diagrama a elaborar

Diagrama de Clases UML

- ¿Para qué sirve?
Para representar la estructura lógica del sistema, mostrando cómo las clases principales (Usuario, Historia, Decisión, Progreso, IA Narrativa, Personalización) se relacionan entre sí.

